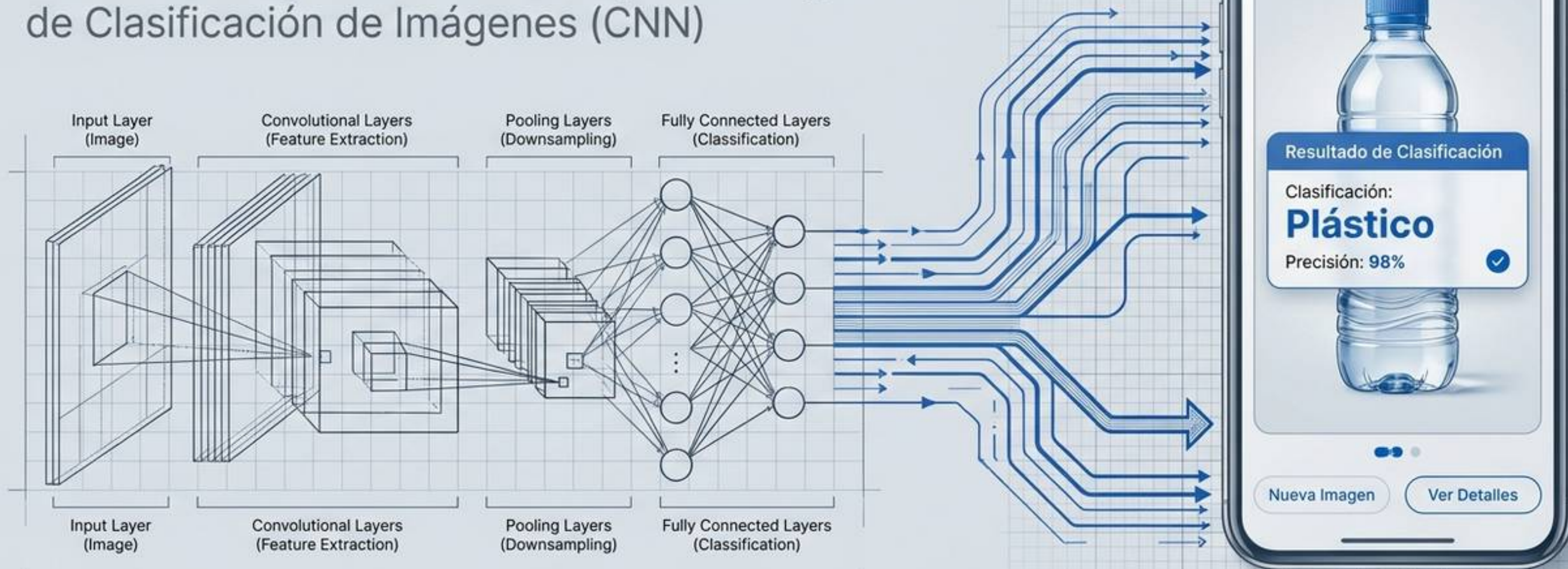
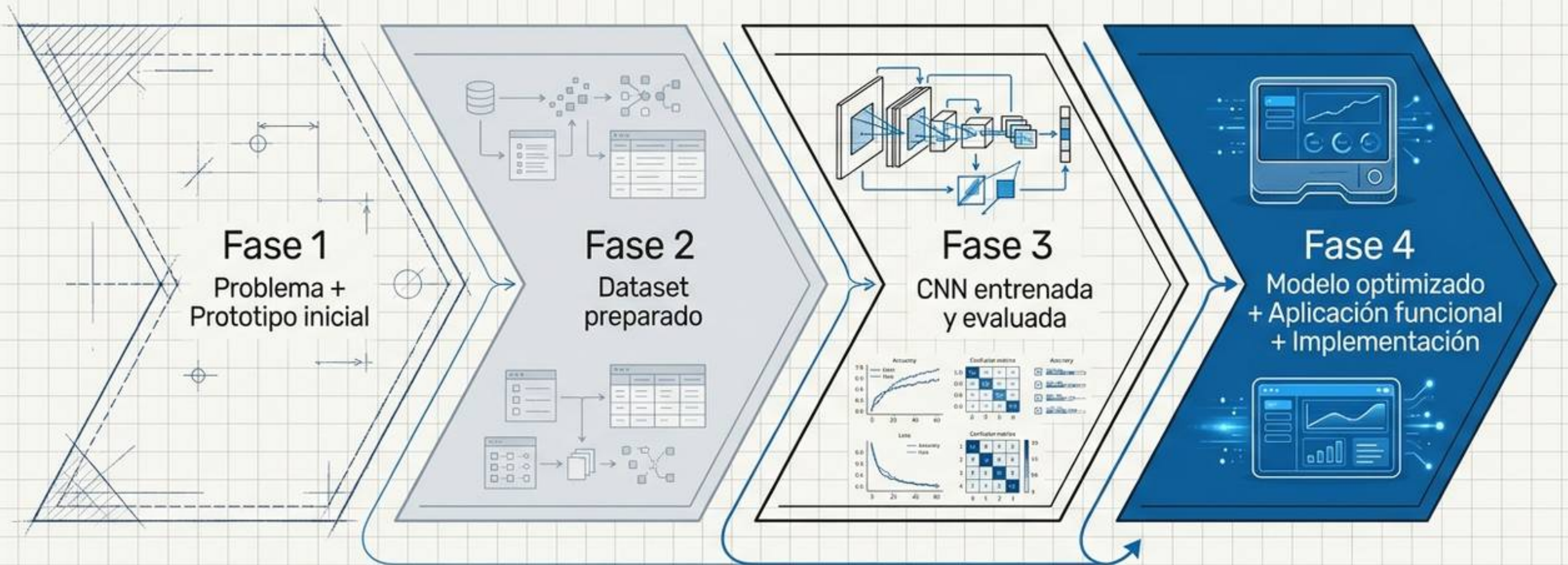


Machine Learning en Producción

Presentación y Defensa del Proyecto Integrador de Clasificación de Imágenes (CNN)

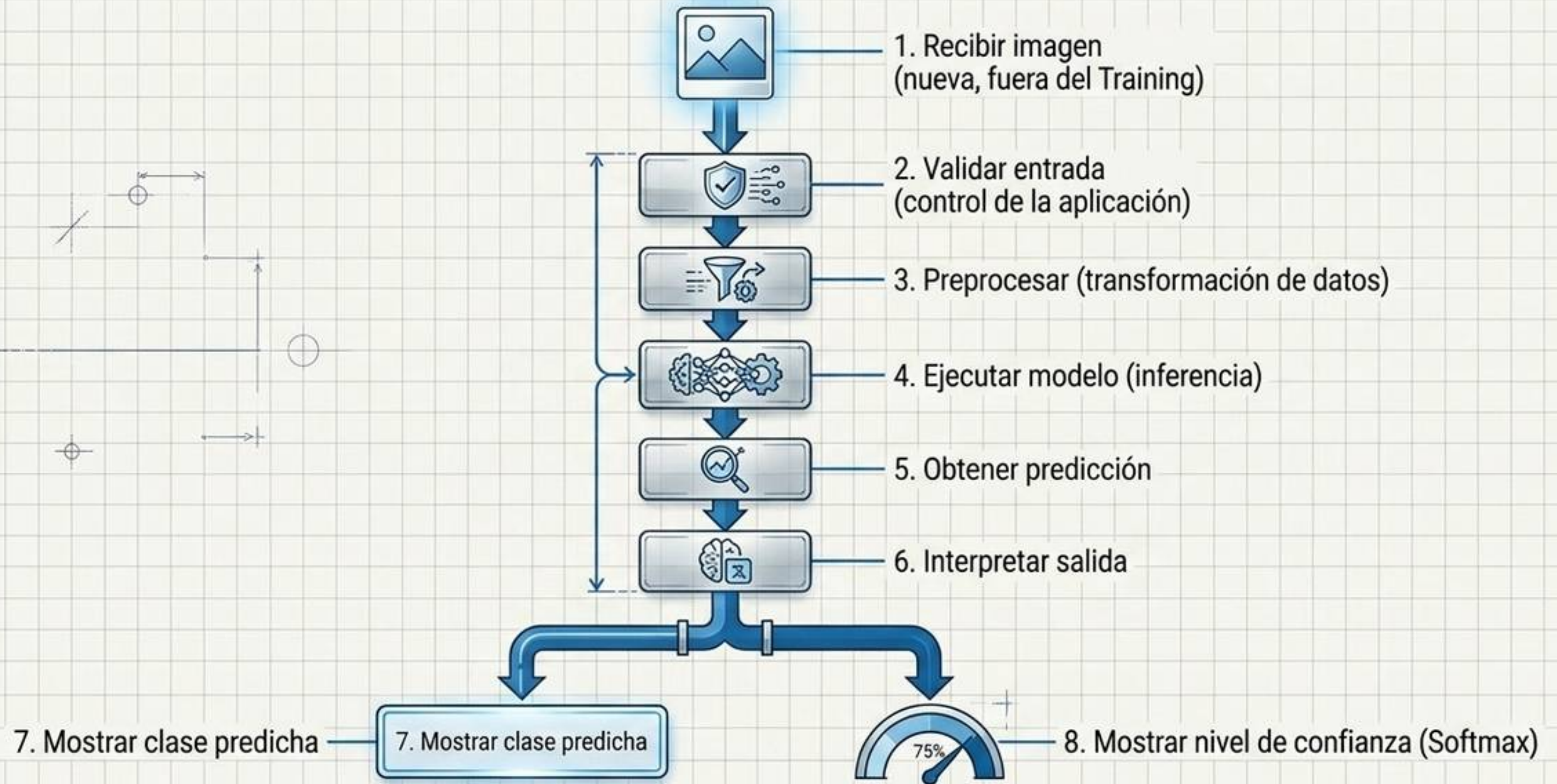


Evolución de la Solución: De la Idea al Producto



El objetivo central es demostrar que el sistema funciona de extremo a extremo, independiente del entorno de desarrollo.

Arquitectura Funcional: Inferencia de Extremo a Extremo



Los 10 Componentes de una Defensa Técnica

1



Problema

¿Qué clasificamos?

2



Dataset

¿Qué imágenes usamos?

3



Modelo

¿Qué CNN seleccionamos?

4



Entrenamiento

¿Cómo se entrenó?

5



Evaluación

¿Qué resultados obtuvo?

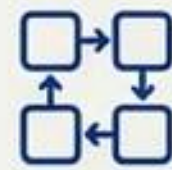
6



Optimización

¿Qué mejoró la eficiencia?

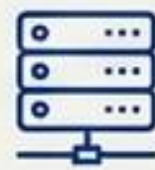
7



Aplicación

¿Cómo se integró?

8



Despliegue

¿Dónde se ejecuta?

9



Demostración

¿Cómo funciona en vivo?

10



Conclusiones

Limitaciones y futuro

Definición del Contexto y los Datos

El Problema

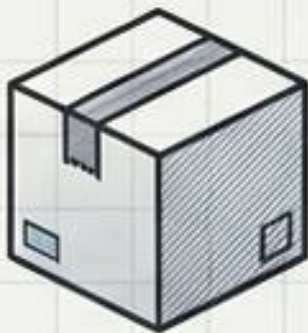
- Aprendizaje Supervisado
- Clasificación multiclase (Deep Learning / CNN)
- Clases:



Plástico



Vidrio

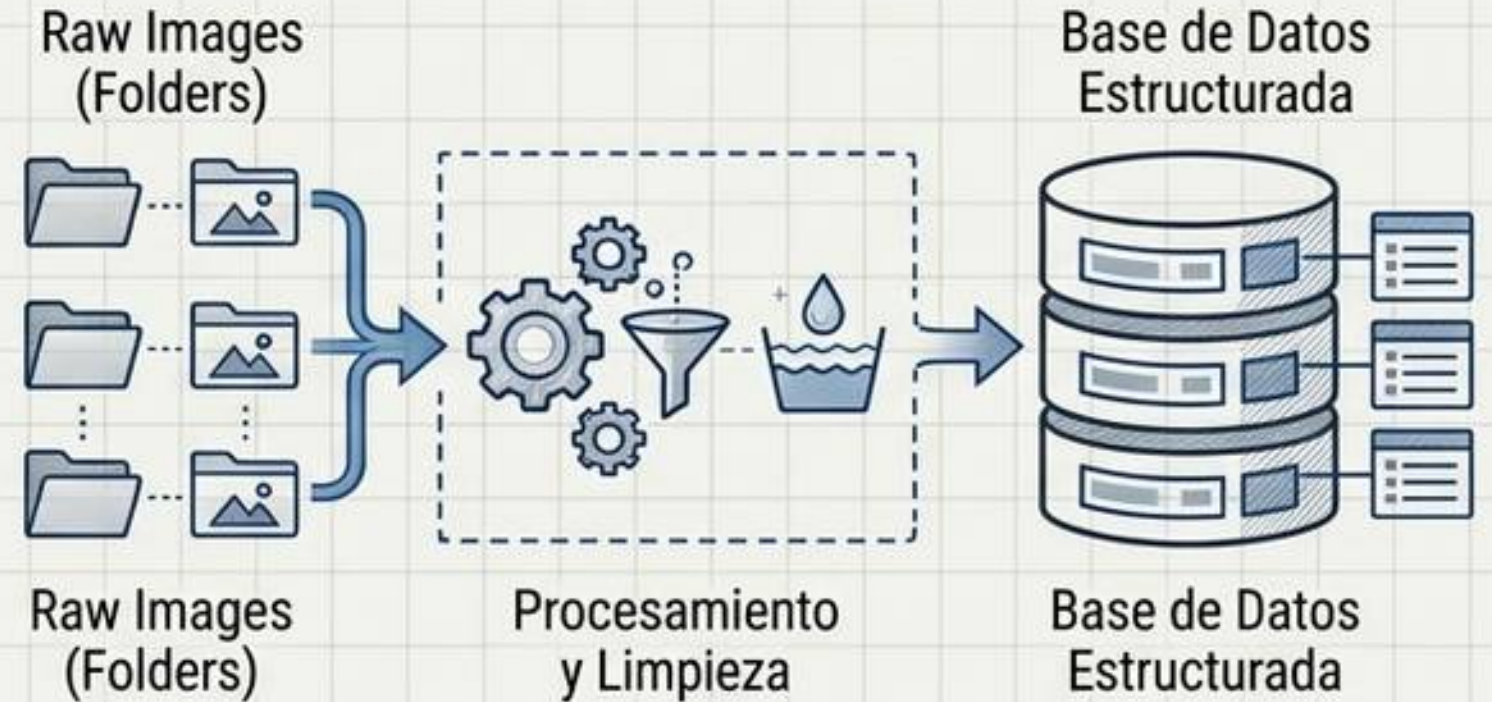


Cartón



Metal

El Dataset Definitivo

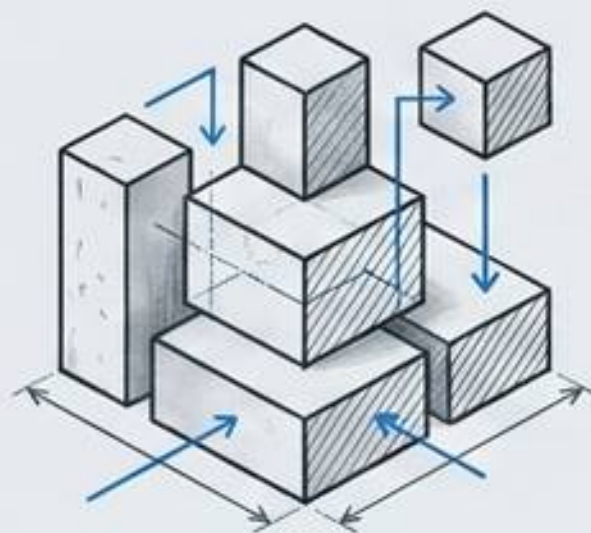


Atributos: Origen, limpieza, etiquetado, balance y resolución.

Técnica clave: Uso de Data Augmentation para robustecer la generalización.

Decisiones Arquitectónicas: ¿Desde Cero o Preentrenada?

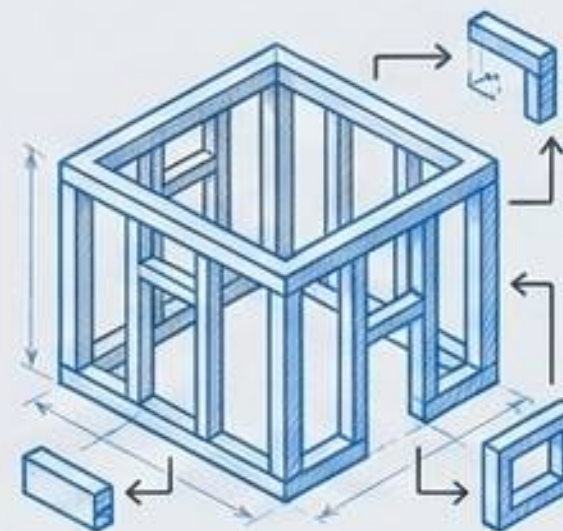
Opción A: CNN desde cero



Mayor control arquitectónico,
pero requiere más datos y tiempo
de Training.

Versus

Opción B: Transfer Learning / Fine-Tuning



Uso de arquitecturas probadas.
Mejor desempeño con Datasets
reducidos y menor consumo de
recursos.

Justificación Basada en Evidencia (Ejemplo):

Se seleccionó MobileNetV2 Fine-Tuned porque presentó el mejor equilibrio: 90% de Accuracy con un tamaño de solo 14 MB



Diagnóstico y Comportamiento del Modelo (Test Set)

Métricas Globales

- Accuracy
- Loss
- Precision
- Recall
- F1-score

(Observar convergencia,
Overfitting o Underfitting)

	Plástico	Vidrio	Cartón	Metal
Plástico	92	2	5	1
Vidrio	3	86	2	9
Cartón	4	1	93	2
Metal	2	8	1	89

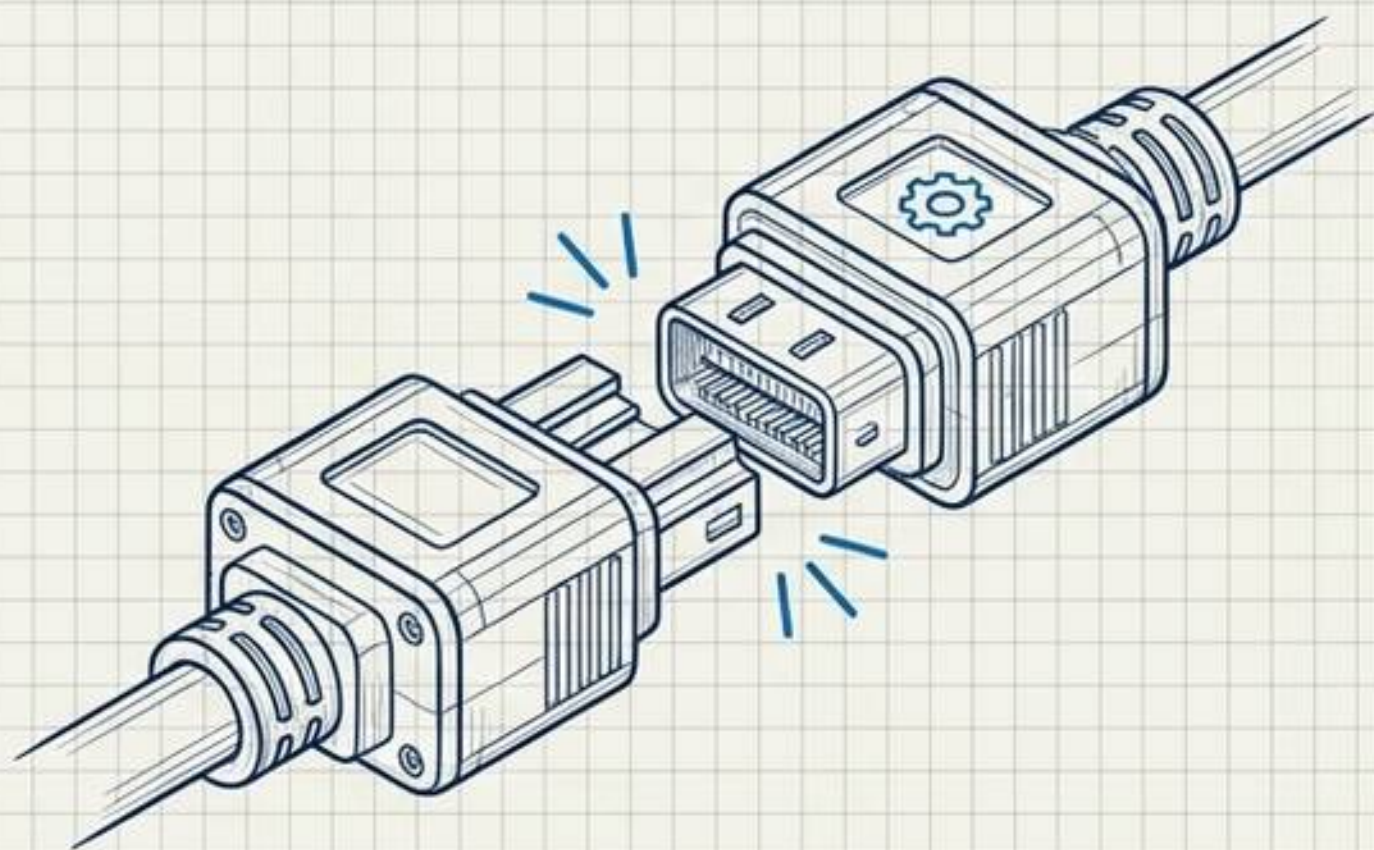
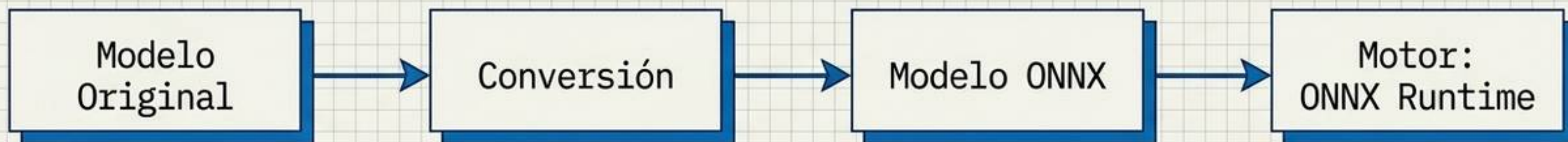
Insight Clínico: El modelo confunde frecuentemente vidrio con metal. Demuestra comprensión del comportamiento real.

El Impacto de la Optimización (Cuantificación y Poda)

Métrica	Original (Baseline)		Optimizado
Accuracy	90,1%		89,6%
Tamaño	84 MB		23 MB
Latencia	240 ms		105 ms

Conclusión Técnica: El compromiso obtenido es exitoso; se acepta una reducción marginal de desempeño (-0,5%) a cambio de reducir el tamaño en ~70% y acelerar la inferencia más del doble.

Interoperabilidad y el Contrato del Modelo (ONNX)

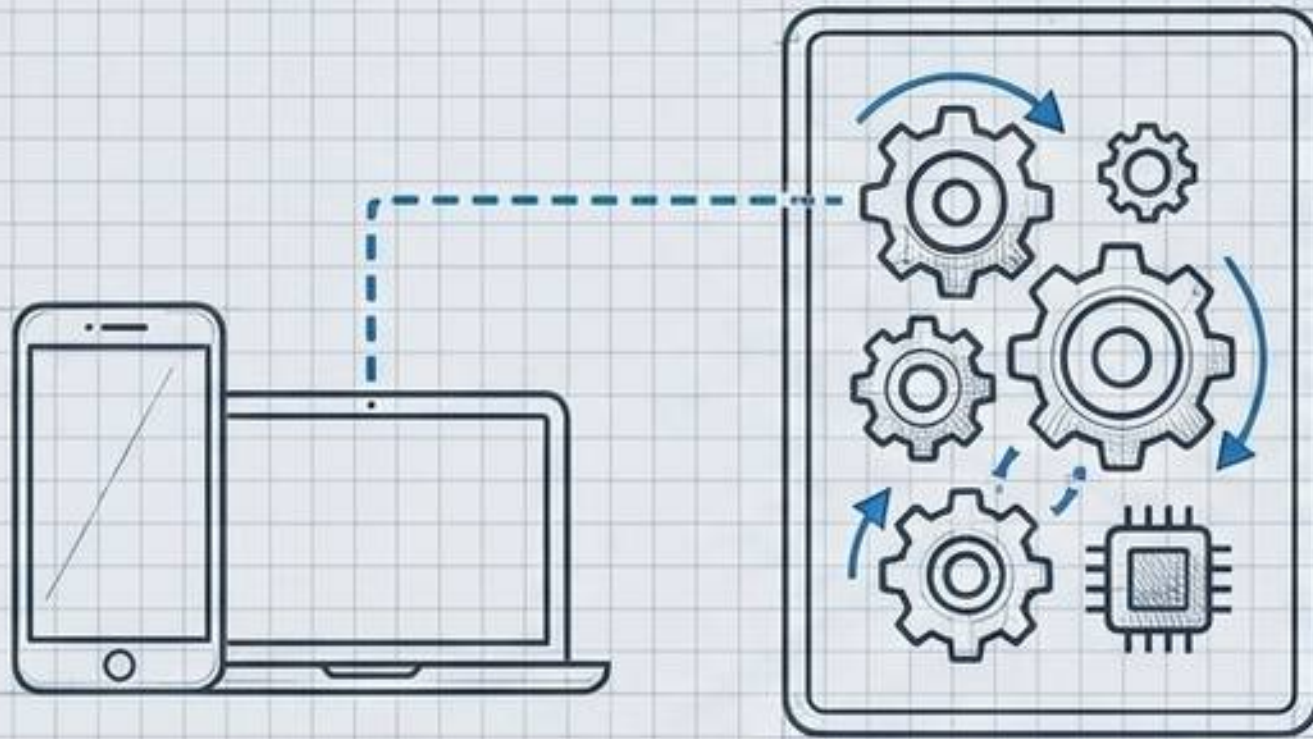


El Contrato Estricto:

- 🔒 **Entrada:** Tamaño y canales esperados (Ej. 224x224 RGB).
- 🔒 **Preprocesamiento:** Mapeo exacto de la normalización del Training.
- 🔒 **Salida:** Vector de probabilidades.
- 🔒 **Orden:** Mapeo exacto del índice con la clase.

Arquitectura de la Aplicación: Entorno de Despliegue

Despliegue LOCAL



Flujo: Usuario -> Interfaz -> Modelo (en dispositivo).

Justificación: Funciona offline, garantiza privacidad de datos, elimina latencia de red.

Despliegue CLOUD



Flujo: Cliente -> Internet (API/Endpoint) -> Servicio Cloud -> Modelo.

Justificación: Escalabilidad masiva, modelo centralizado, menor carga de hardware para el cliente.

Estrategia de la Demostración en Vivo

Regla de Oro:
Entrada real -> Inferencia real -> Salida real
(Imágenes nuevas, no vistas en el Dataset).



Múltiples clases

(probar varias categorías para descartar sesgos)



Baja confianza

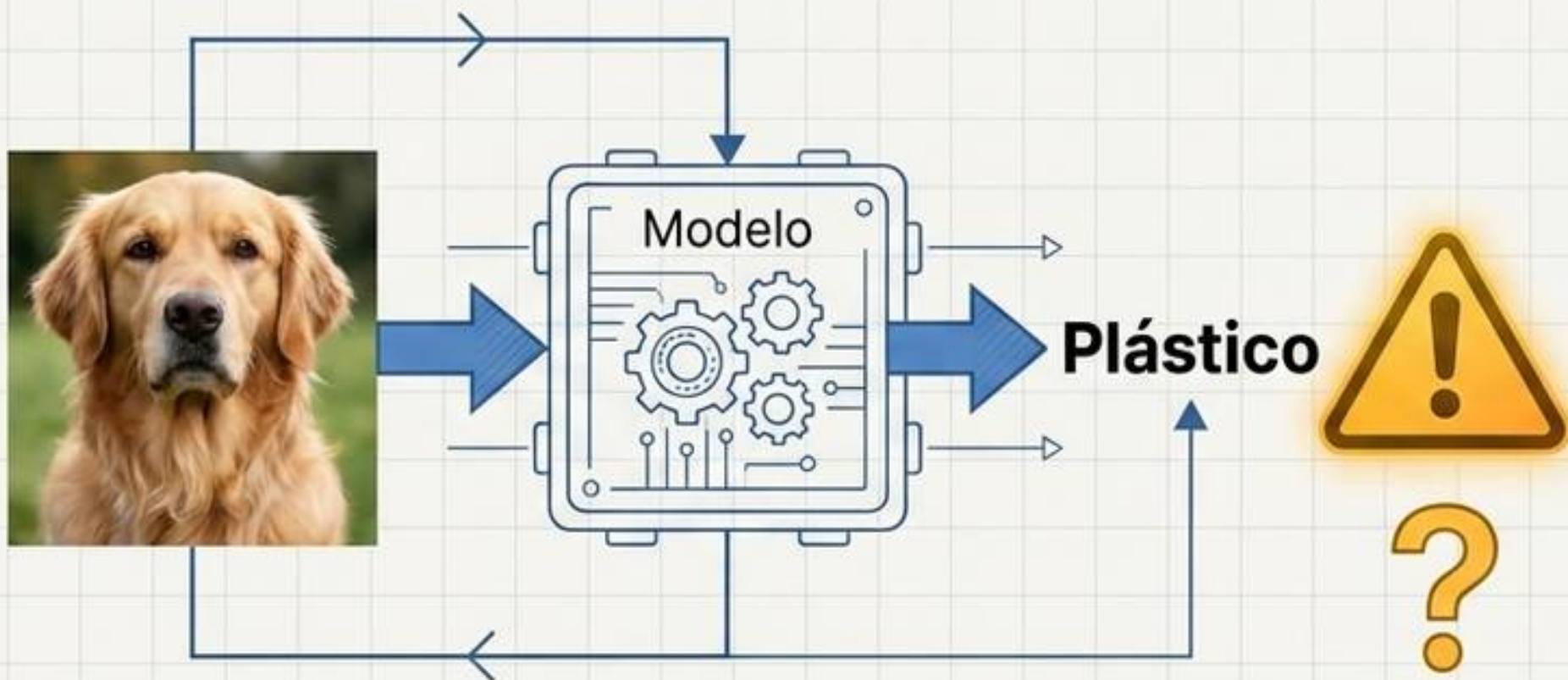
(imagen difícil, ej. 52% de certeza)



Entrada inválida

(ej. subir un PDF para evidenciar manejo de errores)

Reconocimiento de Limitaciones y Data Drift



Imágenes fuera del dominio

Un clasificador cerrado forzará una predicción (ej. un perro clasificado como residuo).

Errores Operacionales

Un fallo en vivo es una oportunidad analítica (¿Fondo distinto? ¿Mala iluminación?).

Riesgos de Data Drift

Cambios en las cámaras o entornos reales degradarán el modelo con el tiempo, requiriendo monitoreo continuo.

Mantenibilidad, Reproducibilidad e Integración Continua (CI)

requirements.txt

```
numpy==1.23.5  
pandas==1.5.3  
scikit-learn==1.2.2  
tensorflow==2.11.0  
gunicorn==20.1.0
```

✓ **Independencia del Entorno:** Cerrar Notebook -> Reiniciar App -> Cargar Modelo. (No basta con 'funciona en mi computador').

✓ **Dependencias:** Uso estricto de requirements.txt.



✓ **Pruebas Mínimas:** Automatizar validaciones de salida.

Dimensiones de la Evaluación Holística

1. Modelo

¿El desempeño predictivo (Accuracy, Loss) es adecuado para el caso de uso?



2. Eficiencia

¿El consumo de recursos, tamaño y latencia son razonables tras la optimización?



3. Integración

¿La aplicación respeta el contrato del modelo (preprocesamiento y validación)?



4. Operación

¿El sistema maneja errores, excepciones y funciona de forma estable en su entorno de despliegue?



Síntesis: El Fin del Proyecto Integrador

- ✓ Un producto de ML no es solo una CNN entrenada; es un sistema de software integral y reproducible.
 - ✓ Las decisiones de arquitectura, optimización y despliegue deben basarse rigurosamente en evidencia y métricas.
 - ✓ La estandarización (como ONNX) asegura la interoperabilidad entre el entorno de datos y la aplicación.
 - ✓ Reconocer el Overfitting, Data Drift y las limitaciones operacionales demuestra madurez profesional.
-
- ✓ Próximo paso: La transición de este prototipo a un entorno de producción real requerirá escala, monitoreo continuo y seguridad.